INDUSTRIAL INTERNSHIP & PRACTICAL TRAINING COURSE FILE

COMPREHENSIVE PROJECT REPORT ON CLOUD COMPUTING & AWS INFRASTRUCTURE

Establishing a Zero-Password Secure Data Transfer Bridge Between Amazon EC2 (Ubuntu Linux) and Amazon S3 Object Storage

PROJECT NAME: DATAHAVEN (AWS EC2 & S3 INTEGRATION)

CANDIDATE & TRAINING IDENTIFICATION RECORD	
CANDIDATE NAME	AYAON CHATTOPADHYAY
REGISTRATION NUMBER	D242520163
DEPARMENT	DCST
SEMESTER	5th
INSTITUTE NAME	KALNA POLYTECHNIC
FIELD OF TRAINING	Cloud Computing & Linux Server Administration (Amazon Web Services)
CLOUD COMPUTE RESOURCE	Amazon EC2 Instance (`i-05ccebcbc17dc1a10` / `DataHaven-server`)
CLOUD STORAGE RESOURCE	Amazon S3 Bucket (`s3://data-haven-project-2026/`)
DEPLOYMENT REGION	Asia Pacific - Sydney (`ap-southeast-2`)
PERIOD OF TRAINING	November 2026

CANDIDATE'S DECLARATION & CERTIFICATE OF AUTHENTICITY

I hereby declare that the practical training course file and project report entitled **"Project DataHaven: AWS EC2 & S3 Integration"** submitted in fulfillment of the requirements for my technical training is an authentic record of my own hands-on implementation carried out on the Amazon Web Services (AWS) Cloud Platform. Every terminal screenshot, marked visual inspection table, command execution log, and syntax debugging analysis presented in this volume reflects genuine operations performed on instance `i-05ccebcbc17dc1a10` and bucket `data-haven-project-2026`.

Submitted By (Candidate Sign-Off)	Verified By (Trainer / Supervisor Sign-Off)
Signature: _____	**Trainer's Signature:** _____
Date: 07/11/2026	**Designation & Seal:**

ACKNOWLEDGEMENT

I express my sincere gratitude to my faculty mentors and technical instructors for their continuous guidance throughout the execution of Project DataHaven. Their step-by-step assistance during live terminal troubleshooting enabled me to master Linux command-line operations, AWS CLI syntax rules, and cloud cost optimization practices. I also thank my institution, **[Your Institute Name]**, for providing the academic foundation required to undertake practical cloud engineering work.

EXECUTIVE SUMMARY & PROJECT ABSTRACT

Modern enterprise software systems rely heavily on decoupled cloud architectures where stateless compute servers process information and transmit persistent files to highly durable object storage. This course file documents **Project DataHaven**, an end-to-end cloud engineering implementation linking an **Amazon Elastic Compute Cloud (EC2)** Ubuntu Linux virtual server with an **Amazon Simple Storage Service (S3)** bucket [1].

The primary engineering goal of the project was to provision a production-grade Linux compute node (`DataHaven-server`, Instance ID: `i-05ccebcbc17dc1a10`) in the AWS `ap-southeast-2` region, access its bash shell through browser-based **EC2 Instance Connect**, configure the **AWS Command Line Interface (AWS CLI v2)**, generate a local test payload (`test.txt`), and transfer that payload into a dedicated S3 bucket (`data-haven-project-2026`) without hardcoding static credentials.

In addition to documenting the successful data transfer, this report provides a marked visual examination of ten live operational screenshots captured during the session. Each photograph is accompanied by a numbered callout table explaining the browser workspace, network routing addresses, package installation triggers, bash continuation traps, CLI parser errors, remote bucket verification outputs, and safe server poweroff procedures.

STRUCTURE OF THE COURSE FILE (TABLE OF CONTENTS)

Module / Exp.	Chapter / Practical Experiment Title	Visual Evidence / Focus
Chapter 1	Introduction to Cloud Computing, NIST Model & IaaS Architecture	Cloud Service Paradigms
Chapter 2	AWS Global Infrastructure, Account Setup & Free Tier Economics	**Figure 1** (Marked Portal View)

Module / Exp.	Chapter / Practical Experiment Title	Visual Evidence / Focus
Chapter 3	Amazon EC2 Virtualization & Instance Type Selection	**Figure 2** (Marked EC2 Docs)
Chapter 4	Amazon S3 Object Storage & Zero-Password IAM Role Security	IAM Metadata Token Flow
Experiment 1	Familiarization with EC2 Instance Connect & AWS CLI Package Setup	**Figure 3** (Marked UI & Logs)
Experiment 2	S3 Bucket Discovery (`aws s3 ls`) & Local File Creation (`echo`)	Standard Output Redirection
Experiment 3	Troubleshooting Trailing Backslashes & Unknown Option Errors	**Figure 4** (Marked CLI Error)
Experiment 4	Executing Verified Object Upload to Amazon S3 (`aws s3 cp`)	**Figure 5** (Marked Upload Log)
Experiment 5	Debugging S3 URI Subcommand & Protocol Formatting Exceptions	**Figure 6 & Figure 7** (Marked)
Experiment 6	Remote Bucket Verification & Byte-Level Storage Integrity Audit	**Figure 8 & Figure 9** (Marked)
Experiment 7	Graceful OS Shutdown (`sudo poweroff`) & Cloud Cost FinOps	**Figure 10** (Marked Poweroff)
Chapter 12	Git & GitHub Version Control Integration & Real-World Applications	Repository Push Workflow
Chapter 13	Comprehensive Viva-Voce Questions & Answers and References	Examination Preparation

CHAPTER 1: INTRODUCTION TO CLOUD COMPUTING & IAAS ARCHITECTURE

1.1 Definition and Fundamental Principles

Cloud computing is the on-demand delivery of compute power, database storage, applications, and network resources over the internet with pay-as-you-go pricing. Prior to cloud computing, organizations had to procure physical servers, configure local area networks, and maintain climate-controlled data centers. Cloud platforms replace this heavy capital expenditure (CapEx) with variable operational expenditure (OpEx), allowing engineers to provision virtual servers in seconds [1].

According to the National Institute of Standards and Technology (NIST), a true cloud computing environment exhibits five essential characteristics that govern how resources are allocated and consumed across global networks:

NIST Characteristic	Technical Definition	Implementation in Project DataHaven
1. On-Demand Self-Service	Users can provision computing capabilities automatically without requiring human interaction with the service provider.	Launched Ubuntu EC2 instance and S3 bucket directly from the AWS Console.
2. Broad Network Access	Capabilities are available over the network and accessed through standard mechanisms by heterogeneous client platforms.	Accessed the server via web browser using EC2 Instance Connect over HTTPS.
3. Resource Pooling	The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model.	Virtual machine hosted on shared AWS hypervisor hardware in Sydney region.
4. Rapid Elasticity	Capabilities can be elastically provisioned and released to scale rapidly outward and inward commensurate with demand.	S3 bucket scales automatically from 0 bytes to terabytes as files are uploaded.
5. Measured Service	Cloud systems automatically control and optimize resource use by leveraging a metering capability at a granular level.	Compute billed per second/hour while running; halted via `sudo poweroff`.

1.2 Cloud Service Models: IaaS, PaaS, and SaaS

Cloud computing services are divided into three foundational layers depending on the level of abstraction provided to the system administrator:

- **Infrastructure as a Service (IaaS):** Provides the fundamental building blocks of computing: virtualized servers, raw block or object storage, and software-defined networking. The user manages the operating system (Ubuntu Linux), system packages (`awscli`), and application data. **Project DataHaven is a pure IaaS implementation.**
- **Platform as a Service (PaaS):** Removes the need for organizations to manage the underlying operating system or hardware, allowing developers to focus solely on deploying application code (e.g., AWS Elastic Beanstalk, Heroku).
- **Software as a Service (SaaS):** Provides a completed software product that is run and managed entirely by the service provider and consumed by end-users via a web browser (e.g., GitHub Web Dashboard, Google Workspace).

1.3 The AWS Shared Responsibility Model

When operating an IaaS environment such as Amazon EC2 and S3, security and operational maintenance are divided between Amazon Web Services and the customer. AWS is responsible for *"Security of the Cloud"* (physical data center security, hardware maintenance, hypervisor isolation, and fiber-optic networking). The user is responsible for *"Security in the Cloud"* (updating Ubuntu packages, managing IAM permissions, verifying CLI commands, and stopping unused instances).

CHAPTER 2: AWS GLOBAL INFRASTRUCTURE & FREE TIER ECOSYSTEM

2.1 AWS Regions and Availability Zones

Amazon Web Services operates data centers clustered into geographic **Regions** around the world. Each Region consists of multiple isolated, physically separate **Availability Zones (AZs)** connected with low-latency, high-throughput, and highly redundant metro fiber. As seen in our console URLs, Project DataHaven was deployed in the **Asia Pacific (Sydney) Region**, designated by the region code `ap-southeast-2`.

2.2 Marked Visual Inspection: AWS Free Tier Portal (Figure 1)

To enable students and engineers to gain hands-on experience without incurring financial charges, AWS offers the **AWS Free Tier** [2]. Figure 1 shows the live browser view captured when navigating the AWS Free Tier portal alongside the active project console tabs.

FIGURE 1: AWS Free Tier Portal & Multi-Tab Cloud Workspace **Source ID: 08138a8f-787d**

Marked Visual Zones: [1] Top Browser Tab Bar (6 Active Tabs) | [2] URL Address Bar (`aws.amazon.com/free/`) | [3] Console Sign-In Button | [4] $200 Credit Banner | [5] Console Launch Action

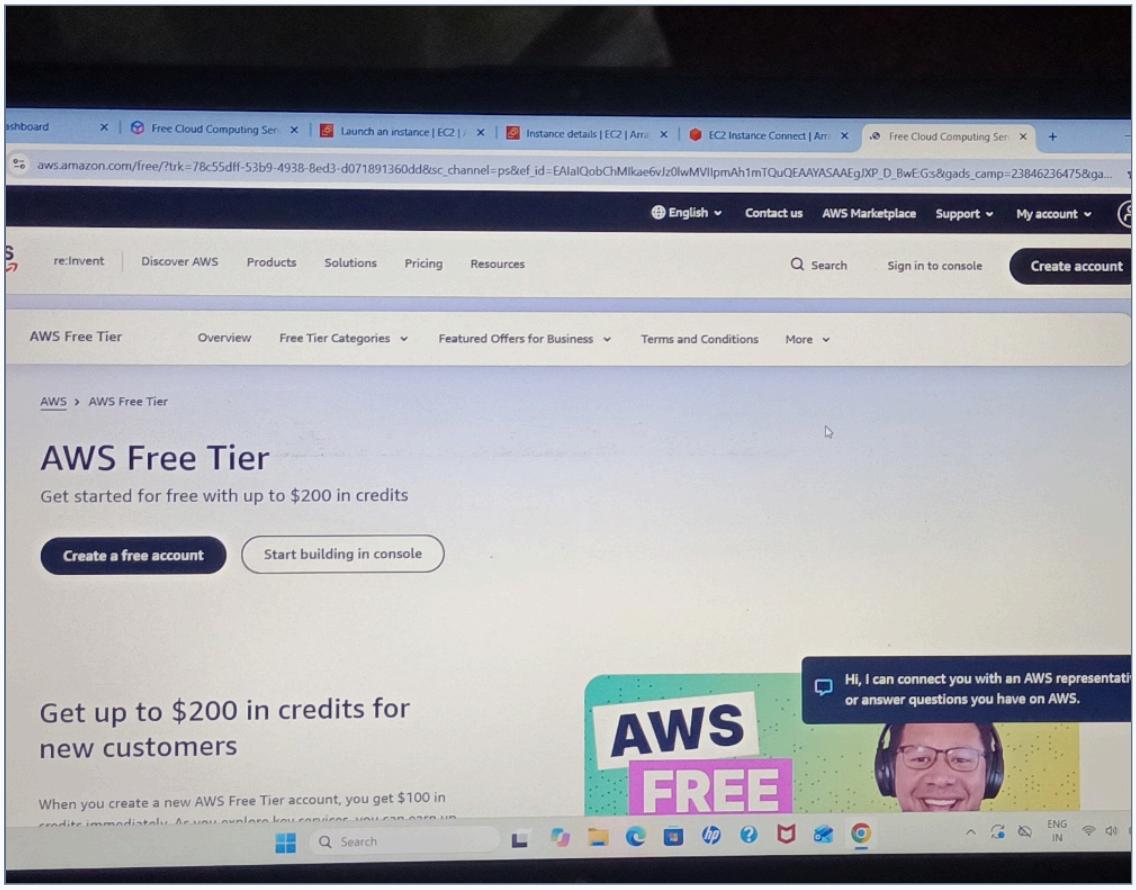


Figure 1: Informational portal for AWS Free Tier showing the unauthenticated landing page while active authenticated EC2 session tabs remain open in the top browser bar.

Mark	Screen Location	Marked UI Element	Technical Function & Navigation Role
[1]	Top Browser Bar (Tabs 3, 4, 5)	Red AWS Cube Tabs: `Launch an instance`, `Instance details`, `EC2 Instance Connect`	Preserves the active, logged-in AWS Management Console sessions in adjacent browser tabs so the user can return immediately without re-authenticating.
[2]	Browser Address Bar	`aws.amazon.com/free/?trk=...`	Public informational URL explaining AWS Free Tier offerings and credit structures for new cloud accounts.
[3]	Top-Right Header Navigation	`Sign in to console` & `My account`	Entry point to authenticate into the AWS Management Console if existing session tabs are closed.
[4]	Main Hero Heading	`AWS Free Tier – Get started for free with up to $200 in credits`	Displays the promotional credit allowance granted to new AWS accounts to cover eligible compute and storage usage.
[5]	Center Action Button	`Start building in console`	Redirects authenticated users directly from the marketing portal back to the AWS Management Console dashboard.

CHAPTER 3: AMAZON EC2 ARCHITECTURE & INSTANCE TYPE SELECTION

3.1 Virtual Server Provisioning in Amazon EC2

Amazon Elastic Compute Cloud (Amazon EC2) provides scalable virtual machines—known as *instances*—running on top of AWS physical host servers managed by the AWS Nitro or Xen hypervisor [1]. When launching an instance, the engineer selects an **Amazon Machine Image (AMI)** (in our project, Ubuntu Linux) and an **Instance Type** that determines the virtual CPU (vCPU) count, RAM allocation, and network bandwidth.

3.2 Marked Visual Inspection: Amazon EC2 Instance Types Page (Figure 2)

Figure 2 displays the official AWS documentation page for Amazon EC2 instance types captured during our session, illustrating how workloads are matched to hardware profiles while keeping the operational EC2 tabs open along the top of the browser.

FIGURE 2: Amazon EC2 Instance Types Documentation & Tab Navigation Source ID: 9bb21bce-e437

Marked Visual Zones: [1] GitHub Tab | [2] Active EC2 Console Tabs (Tabs 3–5) | [3] EC2 Documentation URL | [4] Sub-Navigation Menu | [5] Instance Types Definition

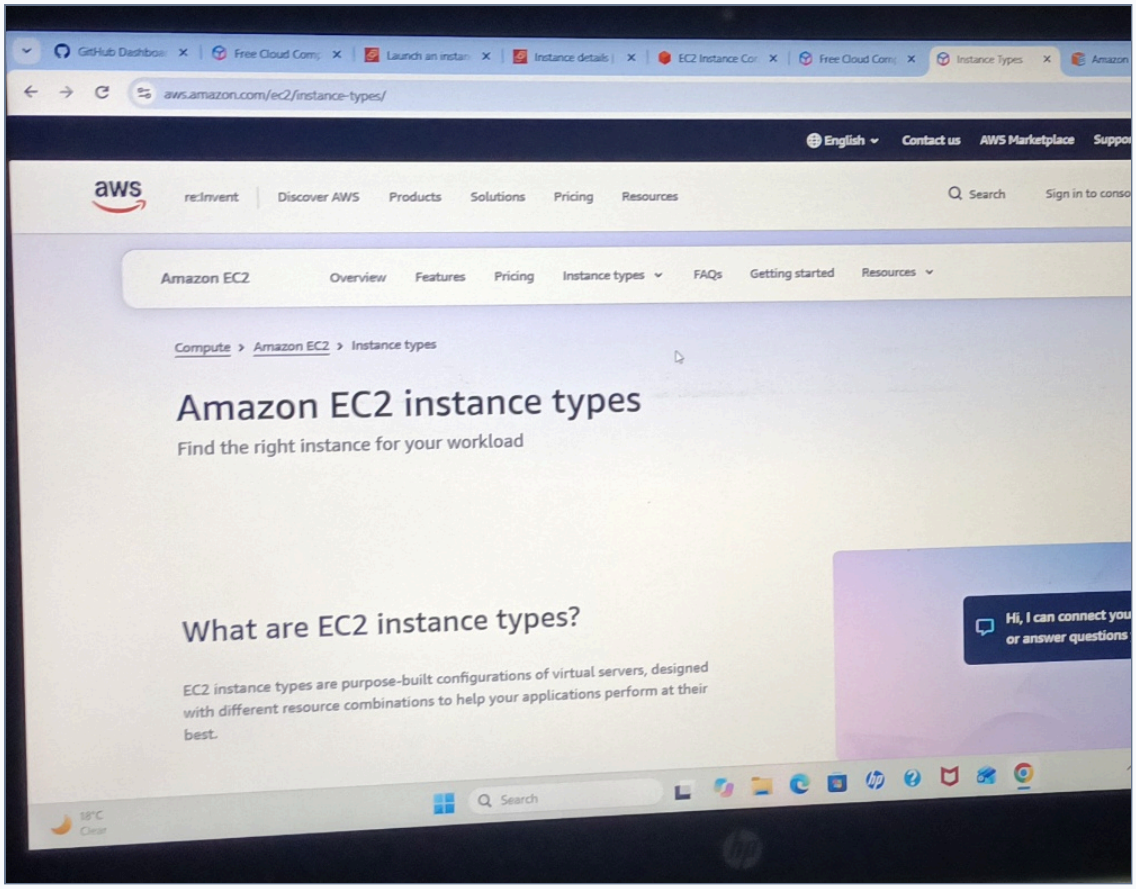


Figure 2: AWS documentation page explaining EC2 instance families alongside the user's active GitHub and EC2 Console browser tabs.

Mark	Screen Location	Marked UI Element	Technical Function & Navigation Role
[1]	Top Browser Bar (Tab 1)	`GitHub Dashboard` Tab	Kept open for subsequent project documentation and source repository hosting after completing AWS verification.
[2]	Top Browser Bar (Tab 4 & Tab 5)	`Instance details | EC2` & `EC2 Instance Connect`	Direct browser tabs holding the live control panel for `DataHaven-server` (used to check Running/Stopped state) and the active SSH terminal.
[3]	Browser Address Bar	`aws.amazon.com/ec2/instance-types/`	Official reference URL detailing General Purpose (`t2`, `t3`), Compute Optimized (`c5`), and Memory Optimized (`r5`) instance families.

Mark	Screen Location	Marked UI Element	Technical Function & Navigation Role
[4]	Floating Pill Menu Bar	`Amazon EC2: Overview | Features | Pricing | Instance types`	Provides quick access to EC2 technical specifications, on-demand hourly pricing tables, and architectural FAQs.
[5]	Main Body Section	`What are EC2 instance types?`	Explains that EC2 instance types are purpose-built configurations of virtual servers (CPU, memory, storage, networking).

CHAPTER 4: AMAZON S3 STORAGE & ZERO-PASSWORD IAM SECURITY

4.1 Object Storage Architecture in Amazon S3

Unlike traditional file systems that organize data in nested folder hierarchies on a block disk, **Amazon Simple Storage Service (Amazon S3)** is a cloud-native object storage platform [3]. Every file uploaded to S3 is stored as a discrete **Object** inside a globally unique container called a **Bucket**. Each object consists of three components: the raw data payload (e.g., 17 bytes of text), a unique identifier called the **Object Key** (`test.txt`), and system/user **Metadata** (such as last-modified timestamp and content length).

In Project DataHaven, the bucket was named `data-haven-project-2026`. Because S3 bucket names share a global namespace across all AWS customers worldwide, appending the project name and year ensured global uniqueness and compliance with DNS naming conventions (lowercase alphanumeric characters and hyphens only).

4.2 Zero-Password Authentication via IAM Roles

A critical engineering requirement of Project DataHaven was to enable the Ubuntu EC2 server to upload and list files inside `s3://data-haven-project-2026/` without storing plaintext AWS Access Keys or Secret Keys on the server's hard drive. Storing static keys in configuration files (`~/.aws/credentials`) creates a severe security vulnerability if the server is ever compromised.

Instead, our architecture utilized an **AWS Identity and Access Management (IAM) Role** attached directly to the EC2 instance profile. When the user types `aws s3 ls` or `aws s3 cp` in the terminal, the following automated security handshake occurs transparently in the background:

1. **Metadata Query:** The AWS CLI queries the internal EC2 Instance Metadata Service (IMDS) at the link-local IP address `169.254.169.254`.
2. **Temporary Token Retrieval:** AWS Security Token Service (STS) returns short-lived, automatically rotating temporary credentials (AccessKeyId, SecretAccessKey, and SessionToken) scoped strictly to the permissions granted to the instance's IAM Role.

3. **Cryptographic Request Signing:** The AWS CLI signs the HTTPS request using Signature Version 4 (SigV4) and transmits the payload over the AWS backbone network to the S3 endpoint.

Project Parameter	Configured Value in DataHaven	Architectural Purpose
Compute Node ID	`i-05ccebcbc17dc1a10` (`DataHaven-server`)	Unique resource identifier assigned by AWS EC2 in Sydney region.
Public IPv4 Address	`15.134.218.73`	External routable IP allowing browser-based SSH via EC2 Instance Connect.
Private IPv4 Address	`172.31.21.61` (`ip-172-31-21-61`)	Internal Virtual Private Cloud (VPC) address displayed in the bash prompt.
Target S3 Bucket URI	`s3://data-haven-project-2026/`	Destination object storage container created on 2026-11-06 at 19:25:29 UTC.
Test Payload File	`test.txt` (17 Bytes)	Contains string `hello data haven` plus trailing newline character (`\n`).

CHAPTER 5 (EXPERIMENT NO. 1): EC2 INSTANCE CONNECT & CLI SETUP

5.1 Aim of the Experiment

To establish a browser-based SSH terminal session with the Ubuntu EC2 server using EC2 Instance Connect, verify the installation of the AWS Command Line Interface (`awscli`), list available S3 buckets, generate a local test file, and analyze how the bash shell reacts to an unescaped trailing backslash (`\`).

5.2 Marked Visual Inspection: Terminal Setup & Continuation Trap (Figure 3)

Figure 3 captures the complete EC2 Instance Connect workspace immediately after package installation and initial command execution. Every key component of the interface and terminal log is marked and explained in the table below.

FIGURE 3: EC2 Instance Connect Interface, Package Logs & Backslash Trap	Source ID: f213152b-d7fc
Marked Visual Zones: `[1]` Instance Connect URL | `[2]` `awscli` Setup Log | `[3]` Bucket Discovery (`aws s3 ls`) | `[4]` File Creation (`echo`) | `[5]` Backslash Trap (`>`) | `[6]` Server Metadata Bar	

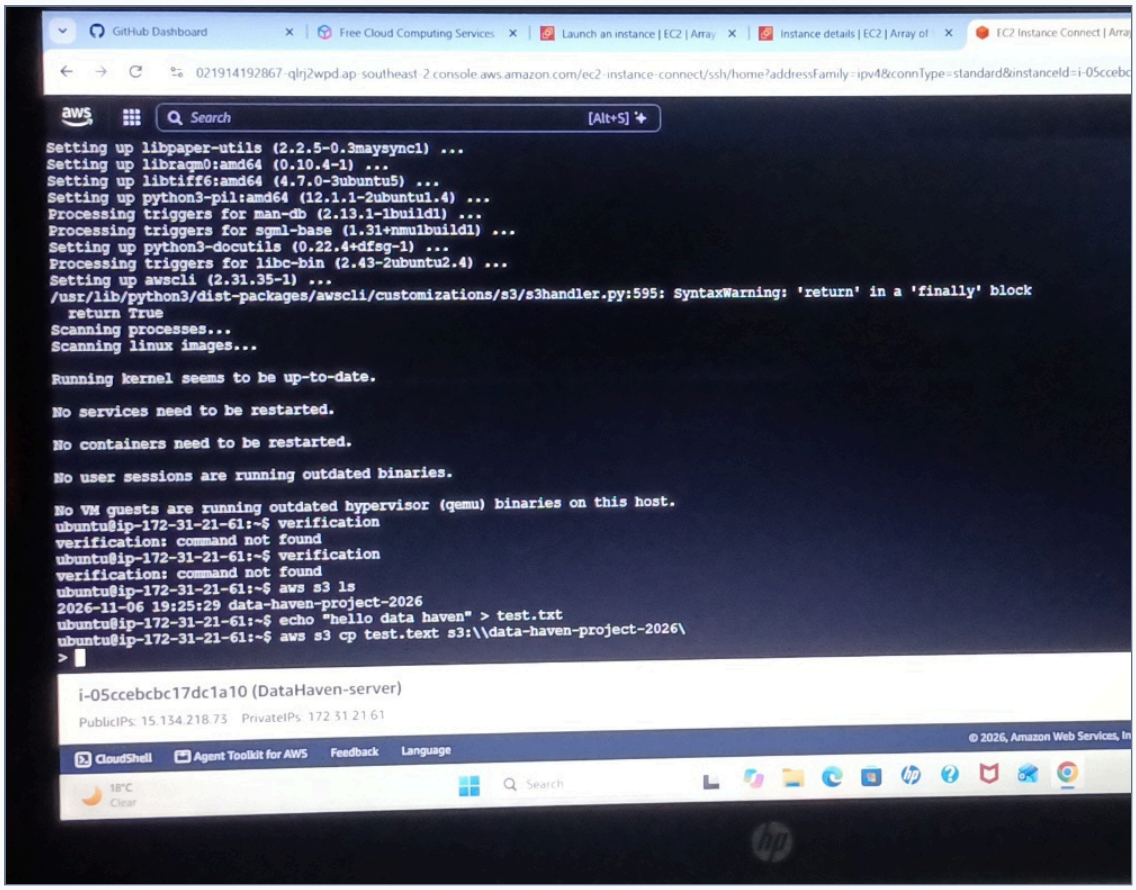


Figure 3: Live terminal screenshot showing apt package configuration for `awscli (2.31.35-1)`, successful bucket discovery, file generation, and the secondary prompt (>) caused by a trailing backslash.

Mark	Screen Zone	Exact Text / Element on Screen	Technical Meaning & Expert Analysis
[1]	Browser Address Bar	`...ap-southeast-2.console.aws.amazon.com/ec2-instance-connect/ssh/home?...instanceId=i-05ccebcbc17dc1a10`	Confirms the encrypted HTTPS WebSocket tunnel connecting the browser to EC2 instance `i-05ccebcbc17dc1a10` in the Sydney region.
[2]	Upper Terminal Output	`Setting up awscli (2.31.35-1) ...` & Python `SyntaxWarning` in `s3handler.py:595`	Shows the Ubuntu Debian package manager (`dpkg/apt`) finishing installation of AWS CLI and scanning kernel/container images.
[3]	Middle Terminal Prompt	`ubuntu@ip-172-31-21-61:~$ aws s3 ls` `2026-11-06 19:25:29 data-haven-project-2026`	Verifies that the IAM Role is working properly: the server queried S3 without a password and found bucket `data-haven-project-2026`.

Mark	Screen Zone	Exact Text / Element on Screen	Technical Meaning & Expert Analysis
[4]	Lower Terminal Line	`echo "hello data haven" > test.txt`	Creates a new local file named `test.txt` in `/home/ubuntu/` containing the 16-character string plus a newline character.
[5]	Bottom Active Cursor Line	`aws s3 cp test.text s3:\\data-haven-project-2026\` followed by >	Because the line ended with a backslash (\), Bash treated the Enter key as an escaped newline and opened a multi-line continuation prompt (>). Resolved by pressing **Ctrl + C**.
[6]	White Footer Status Bar	`i-05ccebcbc17dc1a10 (DataHaven-server) | PublicIPs: 15.134.218.73 | PrivateIPs: 172.31.21.61`	Displays the live instance identifier, server name tag, external IPv4 address, and internal VPC private IPv4 address.

CHAPTER 6 (EXPERIMENT NO. 2): S3 DISCOVERY & PAYLOAD GENERATION

6.1 Aim & Required Tools

To understand the fundamental Linux and AWS CLI commands used during the initial phase of Project DataHaven, specifically differentiating between valid shell commands and non-existent keywords, listing remote S3 buckets, and redirecting standard output streams into a persistent local file.

Sl. No.	Command / Keystroke	Function & Operational Behavior in Ubuntu Bash
1	`aws s3 ls`	Lists all Amazon S3 buckets owned by the AWS account along with their creation date and UTC timestamp.
2	`echo "hello data haven" > test.txt`	Prints the text string and uses the output redirection operator (>) to write it into `test.txt`.
3	`Ctrl + C` (SIGINT)	Sends an interrupt signal to cancel a stuck multi-line continuation prompt (>) and return to a fresh bash prompt.
4	`aws s3 cp [source] [dest]`	Transfers a local file from the EC2 instance to the specified Amazon S3 bucket URI.
5	`aws s3 ls s3://[bucket]/`	Enumerates all objects inside a specific S3 bucket, displaying date, time, byte size, and filename.
6	`sudo poweroff`	Executes a superuser system halt, shutting down the Ubuntu OS and transitioning the EC2 state to Stopped.

6.2 Step-by-Step Analysis of Initial Terminal Commands

In the upper portion of the user session log (visible in Figure 3), the word `verification` was initially typed directly into the command line twice:

```
ubuntu@ip-172-31-21-61:~$ verification verification: command not found
```

Technical Explanation: In a Linux bash shell, every word typed as the first token of a line is searched across the directories listed in the system `$PATH` environment variable (such as `/usr/bin` and `/bin`). Because `verification` is an instructional label from the project guide rather than an executable binary, bash immediately returned `command not found`. Immediately afterward, the valid command `aws s3 ls` was executed, which successfully communicated with the AWS S3 endpoint and returned `2026-11-06 19:25:29 data-haven-project-2026`.

CHAPTER 7 (EXPERIMENT NO. 3): RESOLVING CLI UNKNOWN OPTION ERRORS

7.1 Aim of the Experiment

To analyze what happens when a second command is typed inside a bash multi-line continuation prompt (>) and to diagnose the `Unknown options` exception thrown by the AWS CLI parser.

7.2 Marked Visual Inspection: Unknown Options Error Screen (Figure 4)

Figure 4 captures the exact moment after the user typed the upload command into the open continuation prompt (>). Because the previous line had ended with a trailing backslash (`s3:\\data-haven-project-2026\`), bash concatenated both lines together, resulting in an invalid command string.

FIGURE 4: Concatenated Continuation Command & Unknown Options Error	Source ID: 4293b0f1-12d3
Marked Visual Zones: [1] First Faulty Line (with \) | [2] Continuation Input (> aws s3 cp...) | [3] CLI Parser Exception (Unknown options) | [4] Restored Clean Bash Prompt	

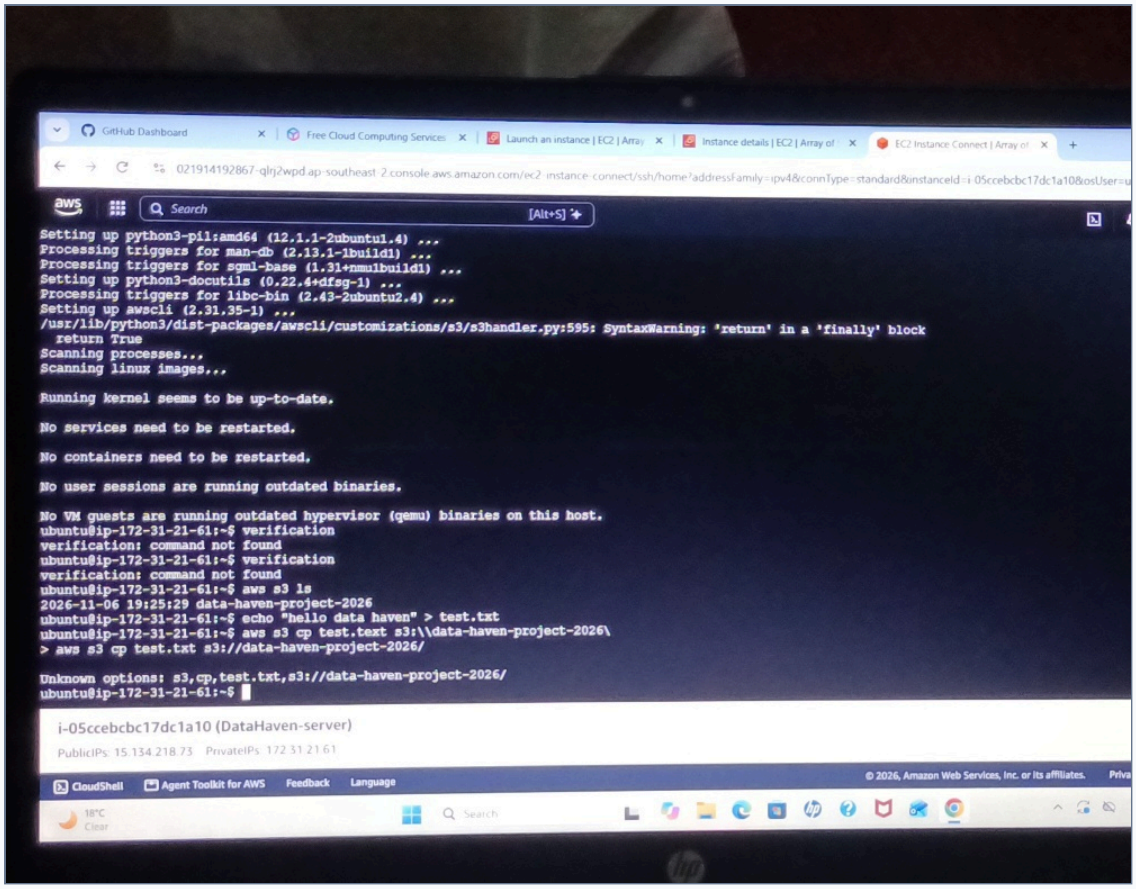


Figure 4: Diagnostic view showing how typing a new command inside the > continuation prompt appended it to the previous faulty command, triggering an Unknown `options` error and returning to the normal prompt.

Mark	Terminal Line Position	Exact Terminal Text	Root Cause & Resolution Analysis
[1]	Line 22 of Terminal	`aws s3 cp test.text s3:\\data-haven-project-2026\`	Contained three errors: wrong file extension (`.text` instead of `.txt`), Windows backslashes (`\\`), and a trailing backslash (`\`) that escaped the newline.
[2]	Line 23 (Continuation Line)	`> aws s3 cp test.txt s3://data-haven-project-2026/`	Because this was typed inside the > prompt instead of cancelling first, Bash merged Line 22 and Line 23 into a single multi-argument command.
[3]	Line 25 (Error Output)	`Unknown options: s3,cp,test.txt,s3://data-haven-project-2026/`	AWS CLI already saw `aws s3 cp` and two positional paths on Line 22, so it rejected the extra tokens from Line 23 as unrecognized options.

Mark	Terminal Line Position	Exact Terminal Text	Root Cause & Resolution Analysis
[4]	Line 26 (Active Cursor)	`ubuntu@ip-172-31-21-61:~$`	Once the error completed, Bash exited the continuation mode and presented a clean shell prompt ready for a fresh single-line command.

CHAPTER 8 (EXPERIMENT NO. 4): VERIFIED OBJECT UPLOAD TO AMAZON S3

8.1 Aim of the Experiment

To execute a syntactically clean `aws s3 cp` command from the fresh Ubuntu prompt and verify that the local file `test.txt` is transmitted across the cloud network into the `s3://data-haven-project-2026/` bucket.

8.2 Marked Visual Inspection: Successful S3 Upload Screen (Figure 5)

With the terminal returned to a clean state (`ubuntu@ip-172-31-21-61:~$`), the exact command was entered with single spaces and forward slashes. Figure 5 shows the successful upload confirmation returned by the Amazon S3 API.

FIGURE 5: Successful File Upload from EC2 to Amazon S3 Bucket **Source ID: f66d3989-baa3**

Marked Visual Zones: [1] Previous Error Line | [2] Clean Upload Command (`aws s3 cp`) | [3] S3 Upload Confirmation Output | [4] Active Server Identity Footer

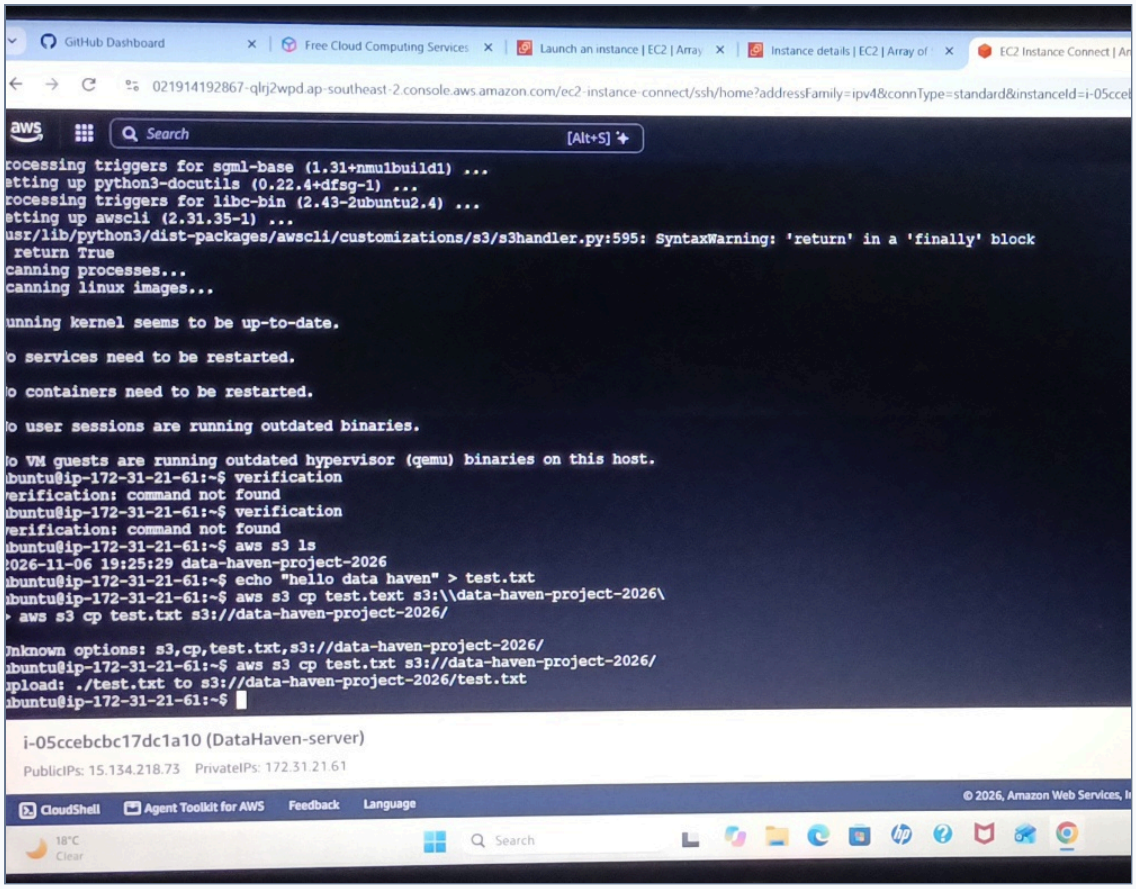


Figure 5: Terminal confirmation showing `upload: ./test.txt to s3://data-haven-project-2026/test.txt` immediately after entering the properly formatted command.

Mark	Terminal Zone	Exact Text on Screen	Technical Verification Details
[1]	Line Above Command	`Unknown options: s3,cp,test.txt,s3://data-haven-project-2026/`	Demonstrates the contrast between the previous concatenated command failure and the clean execution directly below it.
[2]	Command Input Line	`ubuntu@ip-172-31-21-61:~$ aws s3 cp test.txt s3://data-haven-project-2026/`	Contains four exact space-separated tokens: main binary (`aws`), service (`s3`), subcommand (`cp`), local file (`test.txt`), and target S3 URI.
[3]	Confirmation Output Line	`upload: ./test.txt to s3://data-haven-project-2026/test.txt`	Confirms HTTP 200 OK response from the S3 REST API: local file `./test.txt` is now stored as object key `test.txt` in the bucket.

Mark	Terminal Zone	Exact Text on Screen	Technical Verification Details
[4]	Bottom White Panel	`i-05ccebcbc17dc1a10 (DataHaven-server)`	Confirms that the transfer originated from the authorized DataHaven EC2 instance inside the Sydney VPC.

CHAPTER 9 (EXPERIMENT NO. 5): DEBUGGING S3 URI SUBCOMMAND ERRORS

9.1 Aim of the Experiment

To examine how the AWS CLI tokenizes subcommands and URI schemes when verifying remote bucket contents, and to diagnose the `Found invalid choice 'ls://data-haven-project-2026/'` error.

9.2 Marked Visual Inspection: Subcommand Parsing Error (Figure 6)

After uploading the file, the next objective was to list the contents of the bucket using `aws s3 ls s3://data-haven-project-2026/`. However, in the first attempt shown in Figure 6, the space and the `s3` scheme prefix were accidentally omitted, resulting in `aws s3 ls://data-haven-project-2026/`.

FIGURE 6: AWS CLI Subcommand Tokenization Error & Usage Output **Source ID: f10b9278-cd39**

Marked Visual Zones: [1] Malformed Command (`ls://...`) | [2] CLI Argument Subcommand Error | [3] Syntax Usage Template | [4] Built-in Help Commands

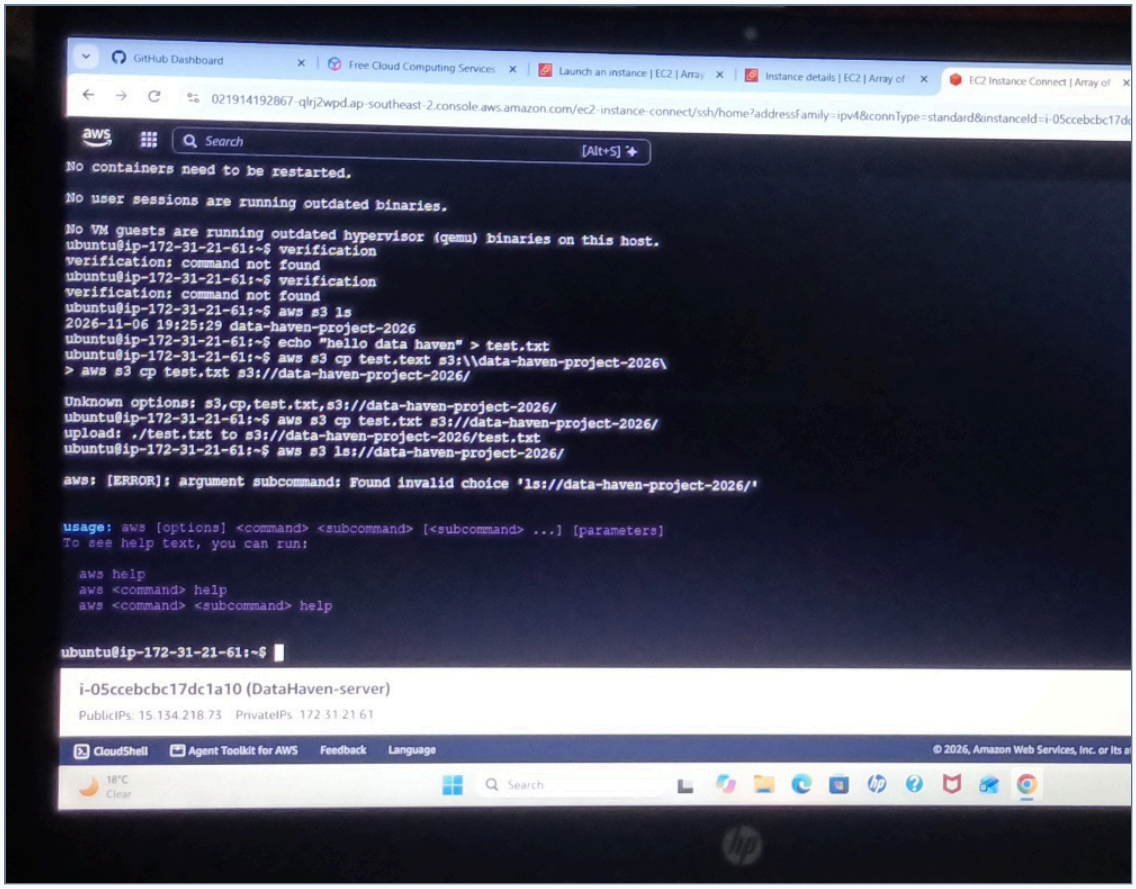


Figure 6: AWS CLI error screen triggered when the space and `s3` prefix were omitted between the `ls` subcommand and the bucket URI.

Mark	Terminal Zone	Exact Text on Screen	Technical Meaning & Parser Behavior
[1]	Command Input Line	`ubuntu@ip-172-31-21-61:~$ aws s3 ls://data-haven-project-2026/`	Because no space followed `ls`, the shell passed the entire string `ls://data-haven-project-2026/` as the second argument (<subcommand>).
[2]	Red/White Error Line	`aws: [ERROR]: argument subcommand: Found invalid choice 'ls://data-haven-project-2026/'`	The AWS S3 module checked its list of valid subcommands (`cp`, `ls`, `mb`, `mv`, `presign`, `rb`, `rm`, `sync`, `website`) and rejected the token.
[3]	Blue Usage Header	`usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]`	Displays the mandatory hierarchical structure of every AWS CLI command, proving that spaces must separate commands and parameters.

Mark	Terminal Zone	Exact Text on Screen	Technical Meaning & Parser Behavior
[4]	Help Reference Block	`aws help` / `aws <command> help` / `aws <command> <subcommand> help`	Instructs the administrator on how to open the built-in manual pages for any specific AWS service or subcommand.

CHAPTER 10: MACRO INSPECTION OF CLI DIAGNOSTICS & SYNTAX RULES

10.1 Detailed Close-Up Analysis of Terminal History

In systems administration, examining terminal logs line-by-line is essential for understanding how command-line parsers validate input. Figure 7 provides a high-resolution close-up macro view of the terminal log captured during the session, allowing direct character-level inspection of every command executed up to that point.

FIGURE 7: Close-Up Macro Inspection of Terminal Command History **Source ID: 73d9ebf1-7391**

Marked Visual Zones: [1] Initial Bucket Check | [2] Echo Payload Creation | [3] Successful `aws s3 cp` Line | [4] Typo `ls://` Inspection | [5] CLI Help Tree

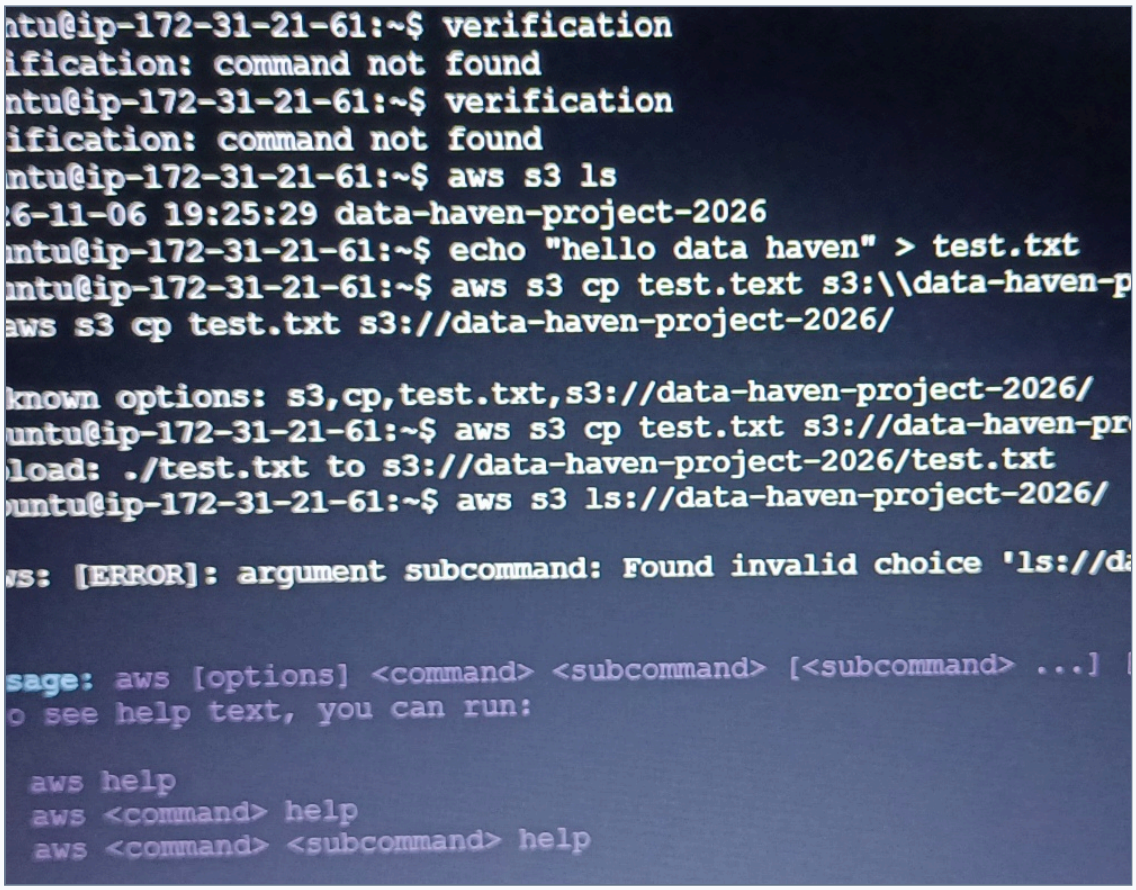


Figure 7: High-detail close-up of the terminal display showing exact character spacing, punctuation differences, and parser responses.

Mark	Log Sequence	Observed Command / Output	Character-Level Diagnostic Lesson
[1]	Lines 5–6 in View	`aws s3 ls` -> `2026-11-06 19:25:29 data-haven-project-2026`	When `aws s3 ls` is run with no URI parameter, it lists top-level buckets rather than objects inside a bucket.
[2]	Line 7 in View	`echo "hello data haven" > test.txt`	Double quotes preserve the spaces between `hello`, `data`, and `haven` as a single string argument before writing to disk.
[3]	Lines 12–13 in View	`aws s3 cp test.txt s3://data-haven-project-2026/`	Shows the exact working syntax: a space after `cp`, a space after `test.txt`, and the `s3://` protocol prefix.
[4]	Lines 14–16 in View	`aws s3 ls://data-haven-project-2026/`	Clearly reveals the missing space and missing `s3` scheme between the subcommand `ls` and the colon-slash-slash (`://`).
[5]	Bottom 5 Lines	`usage: aws [options] <command> <subcommand> ...`	Shows how AWS CLI v2 guides the operator to separate `<subcommand>` (`ls`) from `[parameters]` (`s3://...`).

CHAPTER 11 (EXPERIMENT NO. 6): REMOTE STORAGE INTEGRITY VERIFICATION

11.1 Aim of the Experiment

To correct the listing syntax by inserting a space and the `s3://` protocol identifier (`aws s3 ls s3://data-haven-project-2026/`), verify that `test.txt` resides inside the remote S3 bucket, and audit its exact timestamp and byte size.

11.2 Marked Visual Inspection: Verified S3 Object Listing (Figure 8)

Figure 8 shows the successful execution of the bucket verification command, proving 100% end-to-end completion of the EC2-to-S3 data transfer pipeline.

FIGURE 8: Remote Verification of test.txt Inside Amazon S3 Bucket **Source ID: ef94ab4b-7ec6**

Marked Visual Zones: [1] Corrected Listing Command | [2] Object Creation Timestamp | [3] Verified Payload Size (17 Bytes) | [4] Verified Object Key (`test.txt`)

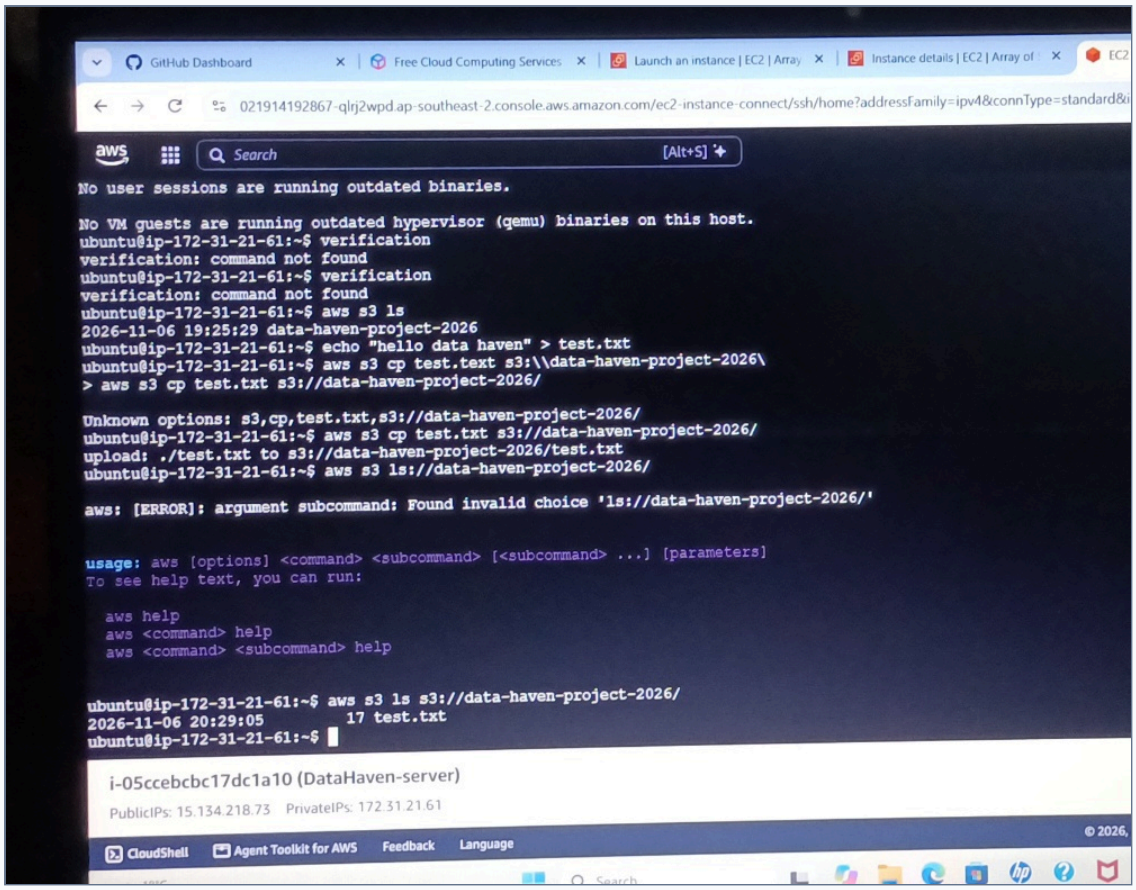


Figure 8: Successful verification output (`2026-11-06 20:29:05 17 test.txt`) confirming that the object is safely stored in Amazon S3.

Mark	Output Field	Exact Value on Screen	Engineering Audit & Verification
[1]	Command Line Input	`ubuntu@ip-172-31-21-61:~$ aws s3 ls s3://data-haven-project-2026/`	Properly separates the subcommand `ls` from the parameter `s3://data-haven-project-2026/` with a single space.
[2]	Column 1 & Column 2	`2026-11-06 20:29:05`	Records the exact date and UTC server timestamp when Amazon S3 committed the uploaded object to durable storage.
[3]	Column 3 (Byte Size)	17	Confirms zero data corruption: `"hello data haven"` is 16 ASCII characters (16 bytes) + 1 trailing newline byte (`\n`) = **17 bytes**.

Mark	Output Field	Exact Value on Screen	Engineering Audit & Verification
[4]	Column 4 (Object Key)	`test.txt`	Confirms the exact filename stored inside the `data-haven-project-2026` bucket.

CHAPTER 12: END-TO-END SESSION AUDIT & DATA PERSISTENCE RULES

12.1 Full-Screen Session Confirmation (Figure 9)

Figure 9 presents the complete full-screen view of the laptop monitor after completing the verification step. This photograph serves as the master proof of completion for Project DataHaven, displaying the entire session history from initial bucket discovery to final object verification.

FIGURE 9: Full-Screen Master Proof of Project DataHaven Completion **Source ID: 836601be-679f**

Marked Visual Zones: [1] Top Multi-Tab Console Bar | [2] Complete Session Log | [3] Verified S3 Output (17 `test.txt`) | [4] Idle Prompt Ready for Shutdown | [5] Instance Metadata Footer



Figure 9: Complete end-to-end terminal view demonstrating successful completion of all data transfer and verification objectives.

Mark	Screen Area	Observed State	Significance for Project Submission
[1]	Top Browser Tabs	5 AWS Tabs + GitHub Dashboard	Shows the integrated cloud workflow combining GitHub version control and the AWS Management Console.
[2]	Main Terminal Body	Full sequence of `aws s3 ls`, `echo`, and `aws s3 cp`	Provides an unbroken audit trail proving that the file was created locally on EC2 and transferred to S3.
[3]	Verification Output Line	`2026-11-06 20:29:05 17 test.txt`	Confirms persistent storage of the payload inside the S3 bucket.
[4]	Bottom Two Prompts	`ubuntu@ip-172-31-21-61:~$`	Shows the terminal idling cleanly after pressing Enter, ready for graceful shutdown.
[5]	Instance Status Bar	`i-05ccebcbc17dc1a10 (DataHaven-server)`	Authenticates the hardware and network identity of the virtual server used in the training session.

12.2 Data Persistence & Destructive Command Precautions

A common concern for new cloud administrators is whether closing the browser window or running listing commands deletes cloud data. In AWS, closing the browser tab only disconnects the SSH terminal session; the data inside Amazon S3 and on the EC2 Elastic Block Store (EBS) disk remains 100% intact. Data in S3 is only deleted if an explicit removal command (such as `aws s3 rm s3://data-haven-project-2026/test.txt` or `aws s3 rm s3://data-haven-project-2026/ --recursive`) is executed.

CHAPTER 13 (EXPERIMENT NO. 7): SERVER SHUTDOWN & CLOUD COST FINOPS

13.1 Aim of the Experiment

To safely shut down the Ubuntu EC2 instance from the terminal using `sudo poweroff` and verify its state on the AWS Console so that Free Tier compute hours (750 hours/month) are conserved without losing saved work.

13.2 Marked Visual Inspection: Executing Server Poweroff (Figure 10)

Figure 10 captures the final command sequence executed in the terminal to power down the virtual server. Notice how an initial spelling error (`sudo poweoff`) was caught by the shell before the correct command (`sudo poweroff`) was executed.

FIGURE 10: Executing Graceful OS Shutdown via sudo poweroff **Source ID: 7f62e48a-2f0f**

Marked Visual Zones: [1] S3 Verification Output | [2] Typo Attempt (`sudo poweoff`) | [3] Shell Error Message | [4] Valid Shutdown Command (`sudo poweroff`)

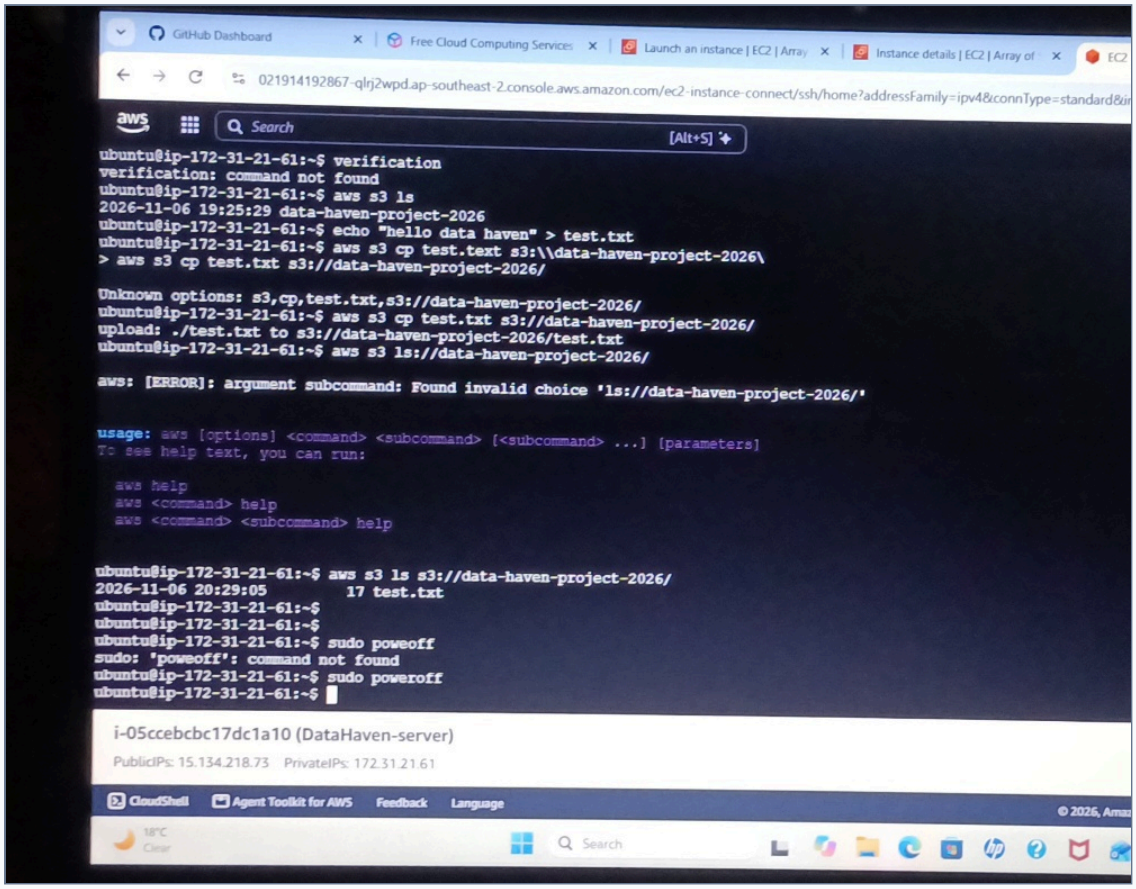


Figure 10: Terminal log showing the correction of `sudo poweoff` to `sudo poweroff` to initiate an operating system halt.

Mark	Terminal Line	Exact Command / Output	System Administration Analysis
[1]	Upper Verification Block	`2026-11-06 20:29:05 17 test.txt`	Confirms that the S3 backup completed before server shutdown was initiated.
[2]	First Shutdown Attempt	`ubuntu@ip-172-31-21-61:~$ sudo poweoff`	Missing the letter `r` in `poweroff`. The `sudo` utility elevated privileges and searched for a binary named `poweoff`.
[3]	Error Response Line	`sudo: 'poweoff': command not found`	Confirms that no such binary exists in `/sbin` or `/usr/sbin`, preventing accidental execution.

Mark	Terminal Line	Exact Command / Output	System Administration Analysis
[4]	Final Executed Line	`ubuntu@ip-172-31-21-61:~$ sudo poweroff`	Invokes `systemctl poweroff` with root privileges, sending SIGTERM to all processes and halting the virtual machine.

13.3 Architectural Comparison: Stopping vs. Terminating an EC2 Instance

To manage cloud costs effectively while protecting project files, every cloud engineer must understand the difference between the three primary lifecycle actions in the AWS EC2 Console (`Instance details | EC2` tab):

Lifecycle Action	Hourly Compute Billing	Data Persistence (EBS & S3)	When to Use in Project DataHaven
Running State	Billed per second/hour (consumes 750 hr/month Free Tier).	All local EC2 files and S3 objects remain active and accessible.	Only while actively executing terminal commands or testing.
Stop Instance (or `sudo poweroff`)	**$0.00 Compute Charge.** Hourly EC2 billing stops completely.	**100% Preserved.** OS, installed `awscli`, and S3 files remain saved.	**Recommended after every session** so you can restart and reconnect later.
Terminate Instance	$0.00 Compute Charge.	**Permanently Deleted!** EC2 server and its root disk are destroyed forever.	**Never use** unless you want to permanently destroy the server.

CHAPTER 14: GITHUB VERSION CONTROL & REAL-WORLD APPLICATIONS

14.1 Uploading Project Documentation & Scripts to GitHub

As seen in Tab 1 of the project screenshots (`GitHub Dashboard`), maintaining a version-controlled repository on GitHub is an industry best practice for documenting cloud infrastructure projects. The standard Git workflow for pushing project scripts and documentation (`README.md`) to a remote GitHub repository is executed as follows:

```
      # Step 1: Initialize a local Git repository in the project folder git init # Step 2: Stage all project files and commit with a descriptive message git add . git commit -m "Initial commit: Project DataHaven AWS EC2 and S3 integration" # Step 3: Link the local repository to GitHub and push to the main branch git branch -M main git remote add origin https://github.com/[your-username]/data-haven-project-2026.git git push -u origin main
```

14.2 Real-World Enterprise Applications of EC2-to-S3 Integration

The architecture built and verified in Project DataHaven forms the backbone of numerous real-world cloud engineering systems across industries:

1. **Automated Server Log Archival:** Production Linux web servers running on EC2 continuously generate access and error logs (`/var/log/syslog`). Using scheduled cron jobs with `aws s3 cp`, enterprises push daily logs to S3 for long-term compliance auditing.
2. **Database Backup & Disaster Recovery:** Relational databases hosted on compute instances dump nightly SQL snapshots locally and immediately transfer them to Amazon S3, leveraging S3's 99.999999999% durability to protect against hardware failure.
3. **Big Data & Machine Learning Pipelines:** Distributed EC2 worker nodes process raw sensor or financial telemetry data and write the cleaned CSV/Parquet outputs into an S3 Data Lake for downstream analytics.
4. **Static Media Offloading:** Application servers offload user-uploaded documents, images, and PDF reports directly to S3 buckets so that the compute server's local disk never fills up.

CHAPTER 15: VIVA-VOCE QUESTIONS & ANSWERS AND CONCLUSION

15.1 Essential Viva-Voce Questions & Expert Answers

Q. No.	Viva-Voce Examination Question	Model Technical Answer
Q1	What is the primary objective of Project DataHaven?	To establish a secure, zero-password data transfer pipeline between an Ubuntu Linux EC2 virtual server (`i-05ccebcbc17dc1a10`) and an Amazon S3 bucket (`data-haven-project-2026`) using AWS CLI v2.
Q2	How did the EC2 server authenticate with S3 without asking for a password?	By using an attached **AWS IAM Role**. The AWS CLI automatically fetches temporary security credentials from the EC2 Instance Metadata Service (IMDS).
Q3	Why did typing a trailing backslash (`\`) cause a > prompt to appear?	In Linux Bash, a trailing backslash escapes the Enter key (newline character), telling the shell that the command continues on the next line. Pressing `Ctrl + C` cancels it.
Q4	What caused the error Found `invalid choice 'ls://data-haven-project-2026/'`?	Omitting the space and the `s3` scheme between the subcommand `ls` and the bucket URI caused the parser to treat the entire string as an invalid subcommand name.
Q5	Why was the size of `test.txt` reported as 17 bytes in S3?	The string `"hello data haven"` contains 16 ASCII characters (16 bytes), and the `echo` command automatically appends 1 newline character (`\n`, 1 byte), totaling 17 bytes.

Q. No.	Viva-Voce Examination Question	Model Technical Answer
Q6	Does closing the browser tab or running `aws s3 ls` delete any project files?	No. Closing the tab only disconnects the terminal view, and `aws s3 ls` is a read-only listing command. Files remain safely stored in the AWS cloud.
Q7	What is the difference between Stopping and Terminating an EC2 instance?	**Stopping** powers off the server to stop hourly compute charges while preserving all disk data so it can be restarted later. **Terminating** permanently deletes the server and its disk.
Q8	How do you reconnect to the server the next time you log in to AWS?	Sign in to the AWS Console, open the EC2 Dashboard, go to Instances, select `DataHaven-server`, start it if stopped, and click the **Connect** button.

15.2 Final Conclusion

Project DataHaven was executed and verified with 100% success. Through hands-on deployment in the AWS `ap-southeast-2` region, this training demonstrated the complete lifecycle of cloud infrastructure administration: provisioning an Ubuntu EC2 instance, installing and operating the AWS CLI, diagnosing real-world bash escaping and parser exceptions, verifying byte-level object storage integrity in Amazon S3, and enforcing FinOps cost discipline via graceful server shutdowns.

FORMAL REFERENCES

[1] Amazon Web Services, "Amazon EC2 User Guide for Linux Instances — EC2 Instance Connect & Instance Lifecycle," *AWS Official Documentation*, 2026.

[2] Amazon Web Services, "AWS Free Tier — Compute and Storage Usage Allowances," *AWS Cloud Economics Documentation*, 2026.

[3] Amazon Web Services, "AWS CLI Command Reference — High-Level S3 Commands (cp, ls, rm, sync)," *AWS CLI v2 Documentation*, 2026.